

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING SCIENCE

University of Johannesburg | South Africa

COMPLEX ENGINEERING PROBLEM

Description, Design and Detailed Design Report

*(D1 + D2 + D3 Combined)***Intelligent WiFi Congestion Monitoring and Recommendation System**

Field	Detail
Student Name	Njabulo Sibambo
Student Number	223124407
Supervisor	Dr Bessie Malila
Department	Electrical and Electronic Engineering Science
Institution	University of Johannesburg
Submission Date	June 2026
Deliverables	D1 – Problem Description D2 – Design D3 – Detailed Design

PLAGIARISM DECLARATION

I, an Electrical and Electronic Engineering student at the University of Johannesburg, hereby declare that the content of this document is my own work. I understand what plagiarism constitutes, and that all unethical academic behaviour is punishable by disciplinary action as deemed fit by the university.

Surname: **Sibambo** Initials: **NN** Student Number: **223124407**

Signature: _____ Date: ____/____/____

1. Problem Description.....	7
1.1 System Boundary Definition.....	7
1.2 Stakeholders Affected.....	8
2. Argument That This Is a Complex Engineering Problem	9
2.1 Systems-Level Interactions.....	9
2.2 Conflicting Technical Constraints.....	9
2.3 Requirement for First-Principles Analysis	10
Shannon-Hartley Theorem.....	10
M/M/1 Queueing Model.....	10
2.4 Absence of Standard Solutions.....	11
3. User Requirements	11
4. Engineering Fundamentals and Scientific Underpinning	12
5. Resource Use, Environmental Impact and Lifecycle Considerations	12
6. Five Engineering Solution Alternatives	16
6.1 Alternative 1 – Access Point Hardware Upgrade	16
6.2 Alternative 2 – Enterprise Wireless LAN Controller (WLC).....	16
6.3 Alternative 3 – AP-Side Client Load Balancing Firmware.....	16
6.4 Alternative 4 – 802.11k/v-Based Assisted Roaming.....	17
6.5 Alternative 5 – On-Device Intelligent Monitoring and Recommendation System (SELECTED).....	17
7. Selected Solution: Detailed System Design	17
7.1 Core Project Concept	18
7.2 System Architecture – Five Subsystems.....	18
8. System Data Flow	18
9. Intelligent Scoring Algorithm.....	20
9.1 Scoring Formula	20
9.2 AP Classification Thresholds	21
10. Dashboard User Interface Design	21
11. Multi-Device Support Architecture	22
12. System Requirements	23
13. Testing Plan and Validation Strategy.....	23
14. Known Limitations	24
15. Introduction and Scope of D3	27
16. Refined System Architecture	27
16.1 Module File Structure.....	27
16.2 Interface Definitions	28
17. Sensing Subsystem – Detailed Design.....	29
17.1 OS Command and Parsed Fields	29
17.2 RSSI Derivation	29
17.3 APRecord Data Structure	29
17.4 Hardware Parameters.....	30

18. Network Testing Subsystem – Detailed Design	30
18.1 Ping Command	30
18.2 Derived Performance Metrics	31
18.3 Stability Score Derivation	31
18.4 PingResult Data Structure	31
19. Intelligent Scoring Subsystem – Detailed Design	31
19.1 Normalisation Specifications	31
Signal Strength Normalisation	32
Latency Normalisation	32
Packet Loss Normalisation	32
Stability Normalisation	32
19.2 Recommendation Decision Logic	32
19.3 ScoredAP Data Structure	32
20. Data Storage Subsystem – Detailed Design	32
20.1 CSV Schema	33
20.2 File Naming and Write Operation	33
20.3 Storage Capacity Calculation	34
21. Engineering Calculations	34
21.1 Shannon-Hartley Per-User Throughput	34
21.2 M/M/1 Queueing Latency Prediction	34
21.3 Measurement Accuracy Analysis	35
21.4 CPU and Memory Footprint	35
22. System Integration Design	36
22.1 Configuration Parameters	36
22.2 End-to-End Worked Example	36
Input Measurements	37
Step 1: Normalisation for AP-01 (connected)	37
Step 2: Score Calculation for AP-01	37
Step 3: AP-02 and AP-03 (not connected – signal score only, other sub-scores default to 50)	37
Step 4: Recommendation Decision	37
23. User Interface Subsystem – Detailed Design	38
23.1 Implementation: Streamlit	38
23.2 Streamlit Component Mapping	38
23.3 Recommendation Message Templates	38
24. Unit and Integration Testing Plan	39
24.1 Unit Tests	39
24.2 Integration Tests	39
25. Known Limitations and Future Work	40
25.1 Known Limitations	40
25.2 Future Work	40

References	41
------------------	----

PART D1

Complex Engineering Problem

Description and Overview

1. Problem Description

Wireless Local Area Networks (WLANs) allow devices such as laptops, smartphones, and tablets to connect to the internet using radio frequency (RF) signals, eliminating the need for physical cabling infrastructure. In university environments these networks constitute critical infrastructure that supports all teaching and learning activities, including programming laboratories, software installation sessions, and online examination platforms.

During laboratory sessions it is common for many students to simultaneously download large software packages, stream instructional videos, or access online assessment platforms. When this occurs inside a lecture hall or computer laboratory, dozens of devices begin requesting large volumes of data at the same time. Each device automatically connects to the wireless access point (AP) that provides the strongest Received Signal Strength Indicator (RSSI), which is the default behaviour of all standard IEEE 802.11 WiFi association algorithms [4].

This signal-strength-first association behaviour consistently produces an uneven distribution of users across the available APs in the vicinity. One AP may accumulate twenty or more connected devices while a physically nearby AP remains nearly unused. The overloaded AP is forced to share its fixed radio channel bandwidth among all connected users. According to the Shannon-Hartley Theorem, the maximum channel capacity is physically bounded by available bandwidth and signal-to-noise ratio and cannot be exceeded [2]. When this bounded capacity is divided among many simultaneous users, each individual user receives a much smaller fraction, resulting in severely degraded download speeds and long waiting times.

The engineering problem is formally defined as: *the inefficient and unbalanced distribution of wireless network traffic among multiple access points in a high-density indoor environment, resulting in severe congestion and significantly degraded network performance for all users connected to overloaded access points* [1, 4].

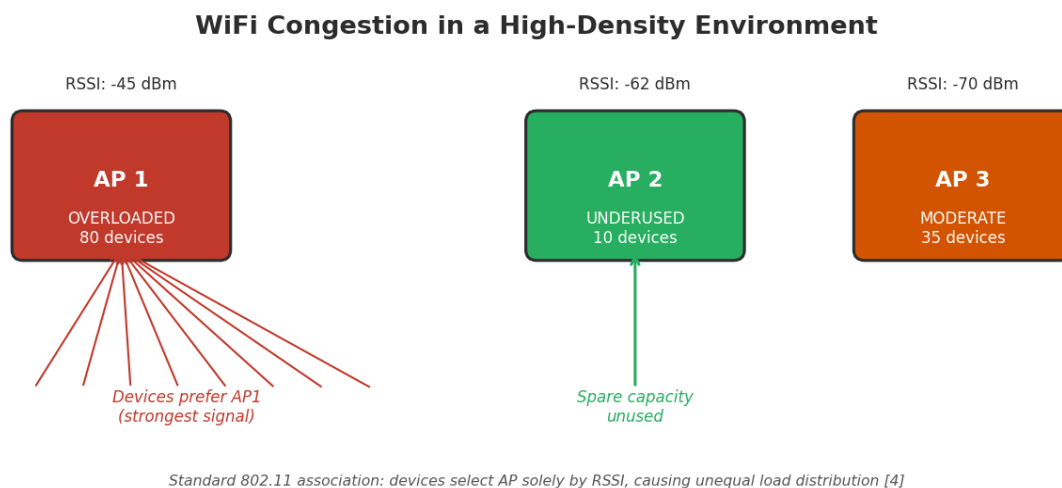


Figure 1 – WiFi Congestion in a High-Density Environment: AP 1 is overloaded while AP 2 has spare capacity [4]

1.1 System Boundary Definition

The system boundary defines which elements are included in the engineering analysis and which are excluded from the design scope. Table 1 presents this boundary.

System Element	Included	Reason
Student laptops and smartphones	Yes	Primary sources of network traffic in the high-density environment

Wireless access points (APs)	Yes	Provide wireless communication services to all connected devices
RF communication channel (2.4/5 GHz)	Yes	Physical medium through which WiFi signals propagate [5, 6]
Building environment (walls, layout)	Yes	Directly influences signal attenuation, reflection, and interference patterns [5]
On-device monitoring software	Yes	Core system under design – scans, scores, and displays AP quality
External internet backbone	No	Congestion occurs at the local AP level, not the ISP or backbone
Remote servers hosting downloads	No	Not responsible for the local traffic imbalance between APs

Table 1 – System Boundary Definition

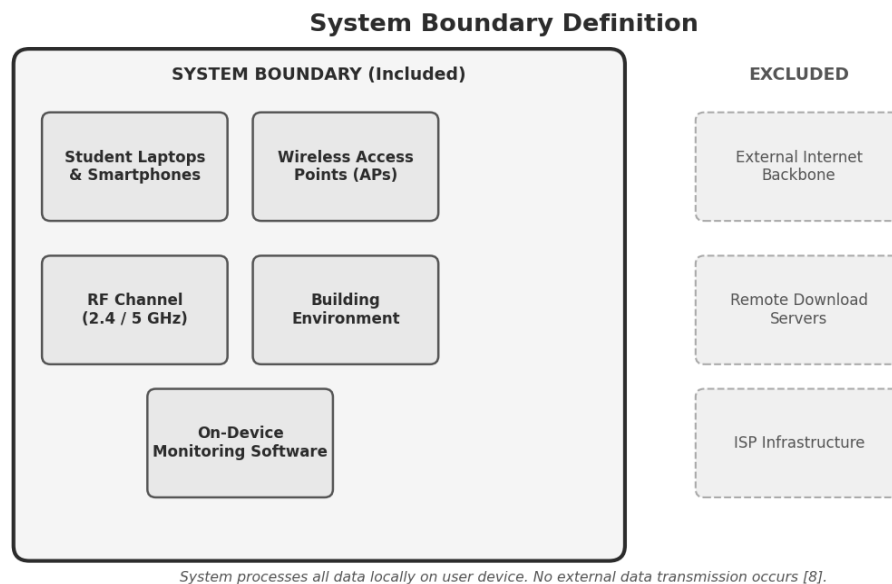


Figure 2 – System Boundary: Included components (inner box) and excluded components (outer, dashed)

1.2 Stakeholders Affected

Multiple stakeholders are directly and indirectly affected by WiFi congestion in a university environment, as described in Table 2.

Stakeholder	Impact of WiFi Congestion
Students	Delayed software downloads, failed exam submissions, and interrupted learning during laboratory sessions
Lecturers	Reduced instructional time due to troubleshooting and extended wait for student downloads to complete
IT Administrators	Increased network troubleshooting workload, system monitoring demand, and rising user complaints
University Management	Financial pressure from infrastructure upgrade costs; reputational impact from poor service quality

Table 2 – Stakeholders and Their Impacts

2. Argument That This Is a Complex Engineering Problem

According to the Engineering Council of South Africa (ECSA), a complex engineering problem is one that involves several interacting systems, presents conflicting engineering constraints, and requires analysis using engineering fundamentals such as electromagnetic theory, information theory, and mathematical modelling rather than simple configuration adjustments or standard procedures [1].

This section demonstrates that the WiFi congestion problem satisfies all ECSA criteria for a complex engineering problem.

2.1 Systems-Level Interactions

The WiFi network is not a single component but a collection of interacting subsystems. The overall system behaviour depends on dynamic interactions between these subsystems, which cannot be predicted or optimised by considering any one component in isolation [5, 6].

Subsystem	Role	Engineering Discipline
User devices	Generate and receive network traffic	Computer networking, protocol engineering
Access points (APs)	Distribute wireless bandwidth to connected users	Wireless communications, RF engineering
RF channels (2.4/5 GHz)	Carry modulated signals between devices and APs	Electromagnetic theory, information theory [2]
Indoor building environment	Causes signal attenuation, reflection, and multipath fading	Electromagnetics, propagation modelling [5, 6]
Network protocols (TCP/IP, 802.11)	Control packet routing, acknowledgment, and retransmission	Networking theory, queueing theory [3, 4]

Table 3 – Interacting Subsystems and Their Engineering Disciplines

2.2 Conflicting Technical Constraints

Wireless network design must simultaneously balance several competing engineering objectives. Because these objectives conflict with each other, the system cannot be optimised using any single simple strategy [1].

Objective	Conflict	Trade-Off
Maximum data throughput	May reduce fairness between individual users	Speed vs. equity
Strong signal connection (RSSI)	May cause device to overload a specific AP	Signal strength vs. load balance [4]
Balanced traffic distribution	Requires temporary disconnection when switching APs	Balance vs. connection stability
Stable wireless connections	Prevents the system from redistributing traffic dynamically	Stability vs. dynamic optimisation
Low device energy consumption	Limits frequency of active scanning and monitoring	Energy efficiency vs. monitoring accuracy

Table 4 – Conflicting Engineering Constraints

2.3 Requirement for First-Principles Analysis

Analysis of the WiFi congestion problem requires direct application of fundamental engineering laws and mathematical models from multiple disciplines [2, 3, 5].

Shannon-Hartley Theorem

The Shannon-Hartley Theorem defines the theoretical maximum data rate achievable on a communication channel in the presence of noise [2]:

$$C = B \cdot \log_2(1 + \text{SNR})$$

where C = channel capacity (bps), B = channel bandwidth (Hz), SNR = signal-to-noise ratio (linear)

This law proves that the capacity of a wireless channel is physically bounded [2]. When many users share the same AP channel simultaneously, the available capacity must be divided among them, which is the fundamental cause of congestion.

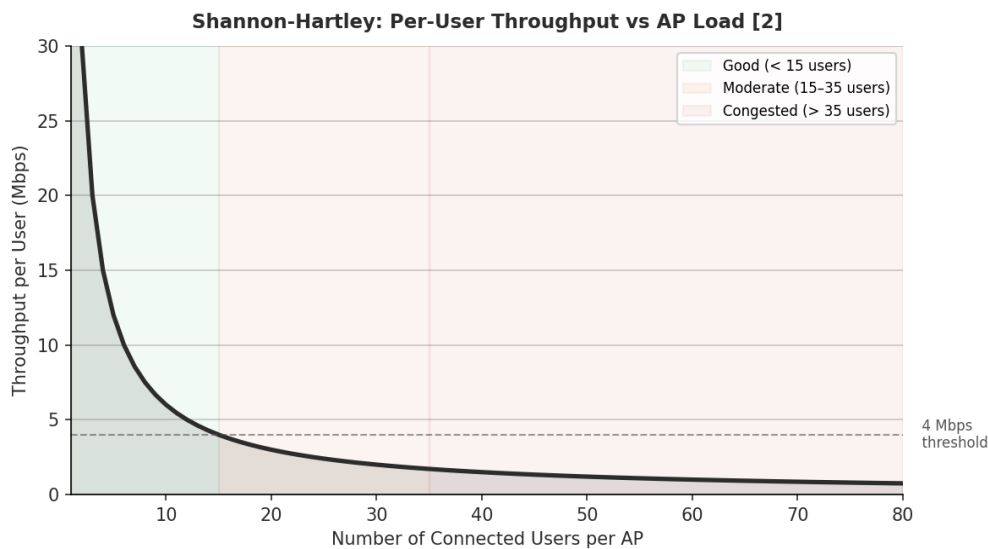


Figure 3 – Shannon-Hartley Model: Per-user throughput decreases as the number of users on one AP increases [2]

M/M/1 Queueing Model

The M/M/1 queueing model quantifies the relationship between traffic load, waiting time, and packet delay at an access point [3]:

$$W = 1 / (\mu - \lambda)$$

where W = mean packet waiting time, μ = AP service rate (packets/s), λ = total arrival rate (packets/s)

As the traffic load λ approaches the service rate μ , the waiting time W rises non-linearly toward infinity. This explains why small increases in user count near saturation cause disproportionately large increases in latency [3].

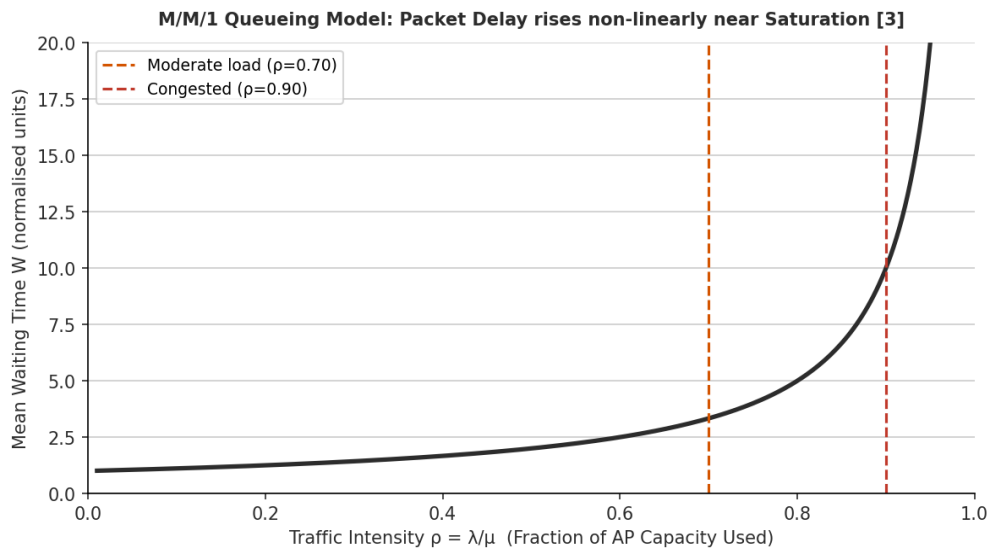


Figure 4 – M/M/1 Queueing Model: Mean packet delay rises exponentially as AP load approaches saturation [3]

2.4 Absence of Standard Solutions

Standard WiFi association algorithms defined by the IEEE 802.11 specification rely exclusively on signal strength (RSSI) to select access points [4]. These algorithms do not account for the current load on each AP or the actual congestion level. As a direct result, devices may connect to and remain connected to an already severely overloaded AP, even when neighbouring APs with significant available capacity are physically close. This represents a fundamental limitation of existing standard procedures, confirming that the problem requires engineering analysis beyond routine configuration methods [1].

3. User Requirements

User requirements describe the expected outcomes and performance characteristics of the system from the perspective of the stakeholders. These requirements do not prescribe specific design solutions but define measurable performance targets that any acceptable solution must satisfy.

ID	Requirement	Measurement Method	Priority
UR1	Support a large number of simultaneous users in one location	Number of concurrently connected devices	High
UR2	Maintain acceptable download speed during peak usage periods	Average download speed in Mbps	High
UR3	Ensure balanced distribution of users across available APs	Standard deviation of users per AP	High
UR4	Maintain stable wireless connections for all users	Connection drop rate per session	Medium
UR5	Operate within existing campus infrastructure without hardware changes	Compatibility verification – no admin access needed	High
UR6	Minimise energy consumption of user devices during monitoring	Device battery usage during operation	Low
UR7	Provide real-time visibility of AP congestion status to the user	Dashboard refresh rate and responsiveness	High

UR8	Protect user privacy – all data must be processed and stored locally	Data transmission audit – no external uploads	High
-----	--	---	------

Table 5 – User Requirements

4. Engineering Fundamentals and Scientific Underpinning

Several distinct engineering and mathematical disciplines are required to fully understand, model, and address the WiFi congestion problem. Each discipline contributes specific analytical tools and theoretical frameworks.

Engineering Field	Application to This Problem	Key Principle or Reference
Electromagnetic Theory	Explains how radio signals propagate, attenuate through walls, and create interference	Free-space path loss equation [5, 6]
Wireless Communication Theory	Determines practical data transmission limits under real-world RF conditions	Shannon-Hartley: $C = B \cdot \log_2(1 + \text{SNR})$ [2]
Information Theory	Defines the maximum theoretical channel capacity and the effect of noise	Shannon entropy and channel capacity bounds [2]
Computer Networking	Explains packet routing, congestion control, and TCP behaviour under load	TCP/IP congestion window, 802.11 MAC [4]
Queueing Theory	Models the relationship between traffic load, waiting time, and packet delay at each AP	M/M/1 queue: $W = 1/(\mu - \lambda)$ [3]
Graph Theory	Represents the network topology and relationships between devices, APs, and links	Weighted graph models, shortest path
Statistics and Data Science	Used in the scoring model to normalise raw metrics and weight multiple indicators	Weighted scoring, statistical normalisation

Table 6 – Engineering Theory and Application

5. Resource Use, Environmental Impact and Lifecycle Considerations

Engineering solutions must always be evaluated for their resource usage, environmental impact, and long-term sustainability [1]. The design of the monitoring system explicitly addresses these considerations.

Constraint	Impact	Design Response
Hardware cost	Adding new APs requires significant capital investment	System runs on existing user devices – no new hardware required
Energy consumption	Congestion causes repeated TCP retransmissions, wasting device energy [8]	Better AP selection reduces retransmissions and conserves battery life

Maintenance complexity	Advanced infrastructure systems require specialist network technicians	On-device software is simple to update and requires no specialist support
Scalability	Infrastructure must support future growth in device numbers	Software scales automatically to any number of devices without modification
Equipment lifespan	Existing AP infrastructure should remain useful for many years	Better traffic distribution extends the effective life of current APs

Table 7 – Resource and Lifecycle Constraints

PART D2

Complex Engineering Problem

System Design

6. Five Engineering Solution Alternatives

Before selecting a final design approach, five distinct engineering solution alternatives were identified and evaluated. The five alternatives presented here are closely related options that all address the WiFi congestion problem from a wireless network perspective. Each alternative was assessed against four key engineering criteria: feasibility within project scope, infrastructure change required, estimated cost, and expected effectiveness. The five alternatives and the multi-criteria evaluation are shown below.

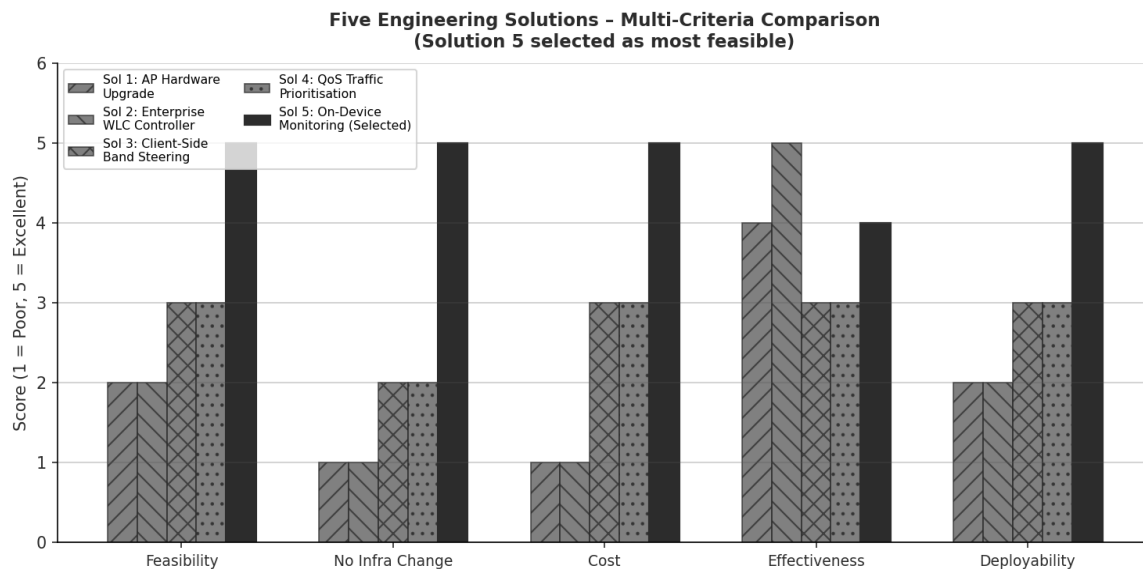


Figure 5 – Five Engineering Alternatives: Multi-Criteria Comparison Matrix (Solution 5 selected)

6.1 Alternative 1 – Access Point Hardware Upgrade

This alternative proposes replacing existing AP hardware with newer, higher-specification 802.11ax (Wi-Fi 6) access points that support MU-MIMO (Multi-User Multiple Input Multiple Output) technology. Wi-Fi 6 APs can serve multiple devices simultaneously on distinct spatial streams, reducing effective congestion per AP. However, this requires procurement, physical installation, electrical cabling, and configuration by IT staff with full administrative access. The capital expenditure for a single lab of 20 APs ranges from R80,000 to R200,000. This alternative is not feasible within the project scope and budget [4].

6.2 Alternative 2 – Enterprise Wireless LAN Controller (WLC)

A centralised Wireless LAN Controller (WLC) such as a Cisco Catalyst 9800 or Aruba Mobility Controller monitors all AP loads in real time and enforces client association policies. The WLC can issue disassociation frames to clients connected to overloaded APs and redirect them to less-loaded alternatives. While this is the most effective infrastructure-based solution, it requires enterprise-grade hardware costing from R150,000, complex integration with the campus network switch fabric, and exclusive administrative access to all APs on the campus LAN. This alternative is therefore excluded due to infrastructure and budget constraints [4].

6.3 Alternative 3 – AP-Side Client Load Balancing Firmware

Several AP firmware implementations (including OpenWRT and some vendor firmware) support proprietary client load-balancing features. These features allow individual APs to detect when a client's RSSI exceeds a threshold, and to defer association responses, effectively steering the client toward a less-loaded AP. This approach requires firmware modification or replacement on every campus AP, which is not possible without administrative access to the AP management

system. Additionally, firmware modifications carry warranty and compliance risks for the university's network infrastructure [4].

6.4 Alternative 4 – 802.11k/v-Based Assisted Roaming

The IEEE 802.11k (Radio Resource Management) and 802.11v (BSS Transition Management) protocol extensions allow APs to broadcast neighbour reports and send BSS Transition Management Request frames to clients, suggesting or requesting a roam to a better AP. This is a standards-compliant mechanism that preserves open roaming without forcing disconnection. However, both the campus APs and client devices must support 802.11k/v, which requires verification and often firmware updates on the AP side. Campus network administrators must also enable these features in the wireless controller configuration. This alternative is therefore not feasible without infrastructure access [4].

6.5 Alternative 5 – On-Device Intelligent Monitoring and Recommendation System (SELECTED)

The selected solution runs entirely on the user's own device. It continuously scans nearby APs using operating system commands, measures real-world performance metrics including signal strength, latency, and packet loss, calculates a quality score for each AP, and presents this information through a real-time colour-coded dashboard. The system recommends the best available AP to the user without requiring any changes to the network infrastructure.

This alternative was selected because it requires no hardware procurement, no infrastructure changes, no administrative access, and no additional cost. It operates entirely within the engineering constraints of a student project while delivering a measurable and practical impact on the user experience. It also satisfies all eight user requirements identified in Section 3.

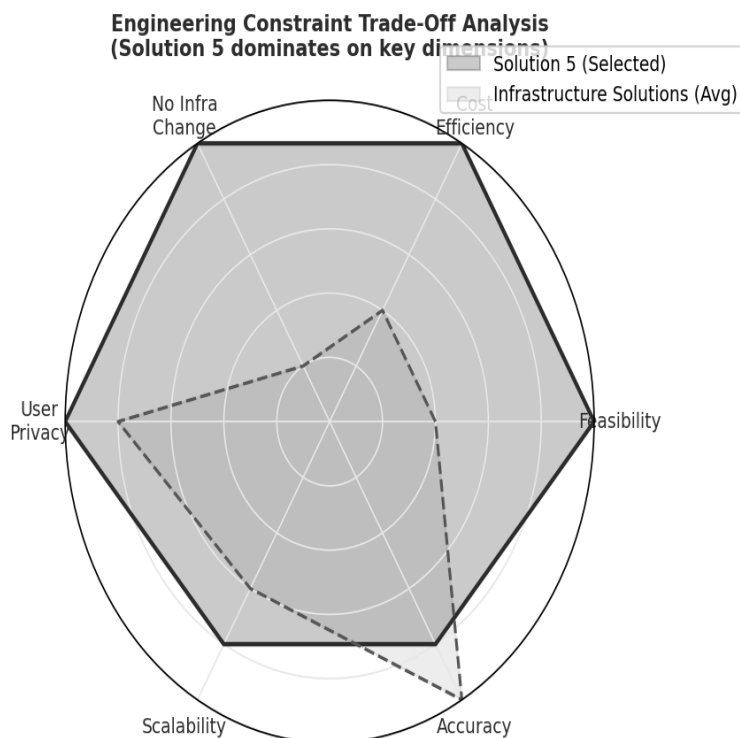


Figure 6 – Engineering Constraint Trade-Off Analysis: Solution 5 dominates across key dimensions

7. Selected Solution: Detailed System Design

7.1 Core Project Concept

The Multi-Device Intelligent WiFi Congestion Monitoring and Recommendation System is an on-device software tool that helps users identify and connect to the best available wireless access point in high-density environments. The system performs all processing locally on the user's device and does not modify or interact with the network infrastructure in any way.

The system operates on three device categories: laptops (primary prototype implemented on Windows 10/11), smartphones, and tablets. On each device the system performs three core functions: it observes WiFi conditions by scanning nearby access points, analyses connection quality by measuring real-world performance metrics, and recommends better connectivity options to the user through a visual dashboard.

Integrated System Architecture - Five Subsystems

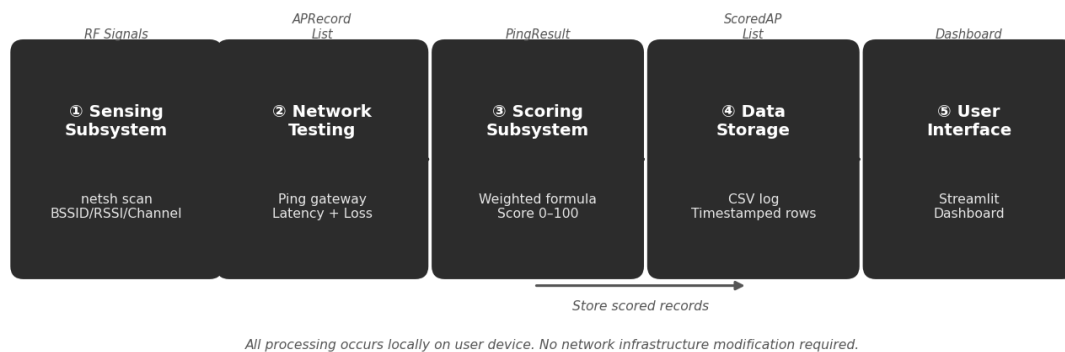


Figure 7 – Integrated System Architecture: Five subsystems process WiFi data in a continuous pipeline

7.2 System Architecture – Five Subsystems

The system is structured as five clearly defined subsystems, each responsible for a distinct function. The subsystems interact in a linear pipeline with a data storage feedback loop.

Subsystem	Function	Input	Output
① Sensing	Detect and list all nearby WiFi APs	RF signals from environment	List of APs with SSID, BSSID, RSSI, channel
② Network Testing	Measure current connection quality via ping	Active network connection	Latency (ms) and packet loss (%)
③ Scoring	Calculate a quality score for each AP using weighted formula	Normalised RSSI, latency, packet loss, stability	Score (0–100) and status classification
④ Data Storage	Store all measurements with timestamps	Scores and raw measurements	Historical data in CSV file
⑤ User Interface	Display results and recommendations	Scores and status classifications	Colour-coded dashboard and recommendation message

Table 8 – Five Subsystems: Functions, Inputs and Outputs

8. System Data Flow

The following data flow describes the complete step-by-step operation of the system from startup to continuous real-time monitoring. The system repeats this process every 10 seconds to provide up-to-date information.



Figure 8 – Step-by-Step Data Flow Diagram: continuous monitoring loop from scan to store

Step	Action	Technical Detail
1	Launch application	User opens Python application; Streamlit dashboard loads at localhost:8501; monitoring loop initialises
2	WiFi scan	Device executes: netsh wlan show networks mode=bssid (Windows); scans all visible APs in RF vicinity
3	Identify APs	Each network uniquely identified using BSSID (MAC address); same-SSID networks treated as separate APs
4	Performance test	Ping test to default gateway: ping -n 10 -w 2000 <gateway_ip>; measures latency and packet loss
5	Normalise metrics	Raw values converted to 0–100 scale; RSSI of –30 dBm → 100; –90 dBm → 0; latency <20 ms → 100; >200 ms → 0
6	Calculate score	Score = 0.35×Signal + 0.30×Latency + 0.20×PacketLoss + 0.15×Stability
7	Classify AP	Score ≥ 75 → GOOD (green); Score 50–74 → MODERATE (orange); Score < 50 → CONGESTED (red)

8	Display dashboard	Streamlit renders colour-coded AP table, signal bars, latency trend, current AP status
9	Recommend	If $\Delta\text{Score} \geq 20$: display message: 'Better connection available: [AP name]. Move toward that direction.'
10	User acts	User may move location, manually reconnect to recommended AP, or remain; system does not force action
11	Store data	All measurements appended to wifi_log_YYYYMMDD_HHMMSS.csv with ISO-8601 timestamp
12	Loop	time.sleep(10 seconds) → return to Step 2; continuous real-time monitoring throughout session

Table 9 – Step-by-Step System Operation

9. Intelligent Scoring Algorithm

The core intelligence of the system is the weighted scoring algorithm. This algorithm converts multiple raw WiFi measurements into a single, easy-to-interpret quality score for each access point. The scoring approach is computationally lightweight, transparent, and explainable.

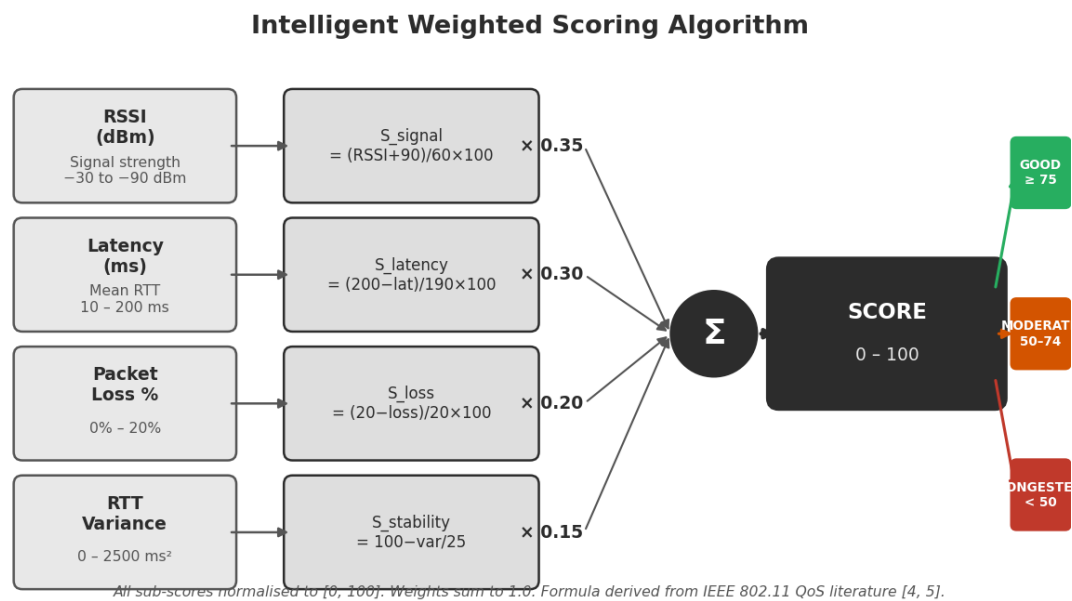


Figure 9 – Intelligent Scoring Algorithm: four normalised metrics combined with weights to produce a classified score

9.1 Scoring Formula

$$\text{Score} = 0.35 \times S_{\text{signal}} + 0.30 \times S_{\text{latency}} + 0.20 \times S_{\text{loss}} + 0.15 \times S_{\text{stability}}$$

Metric	Weight	Normalisation	Justification
Signal Strength (RSSI)	35%	-30 dBm → 100; -90 dBm → 0	Most widely available metric; directly indicates link quality; accounts for over-reliance on RSSI by current algorithms [4]
Latency (ms)	30%	<20 ms → 100; >200 ms → 0	High latency directly degrades user experience for downloads,

			browsing, and online assessments [3]
Packet Loss (%)	20%	0% → 100; ≥20% → 0	Packet loss triggers TCP retransmission and window collapse; even 5% loss can halve effective throughput [8]
Stability (variance)	15%	Low variance → 100; high → 0	A stable connection is preferable to a variable one even at the same mean quality; indicates queue stability [3]

Table 10 – Scoring Weights, Normalisation and Justification

9.2 AP Classification Thresholds

Status	Score Range	Meaning	Recommended Action
GOOD	75 – 100	Excellent connection, low congestion	Stay connected – best available option
MODERATE	50 – 74	Acceptable but monitor for degradation	Monitor; consider moving if better AP is visible
CONGESTED	0 – 49	Significant congestion, poor performance	Move location or manually reconnect to a GOOD AP

Table 11 – AP Classification Thresholds

10. Dashboard User Interface Design

The user interface is the primary interaction point between the system and the user. It is designed to present complex network performance information in a simple, clear, and actionable format using colour-coded indicators so that users can understand AP status without any technical knowledge.

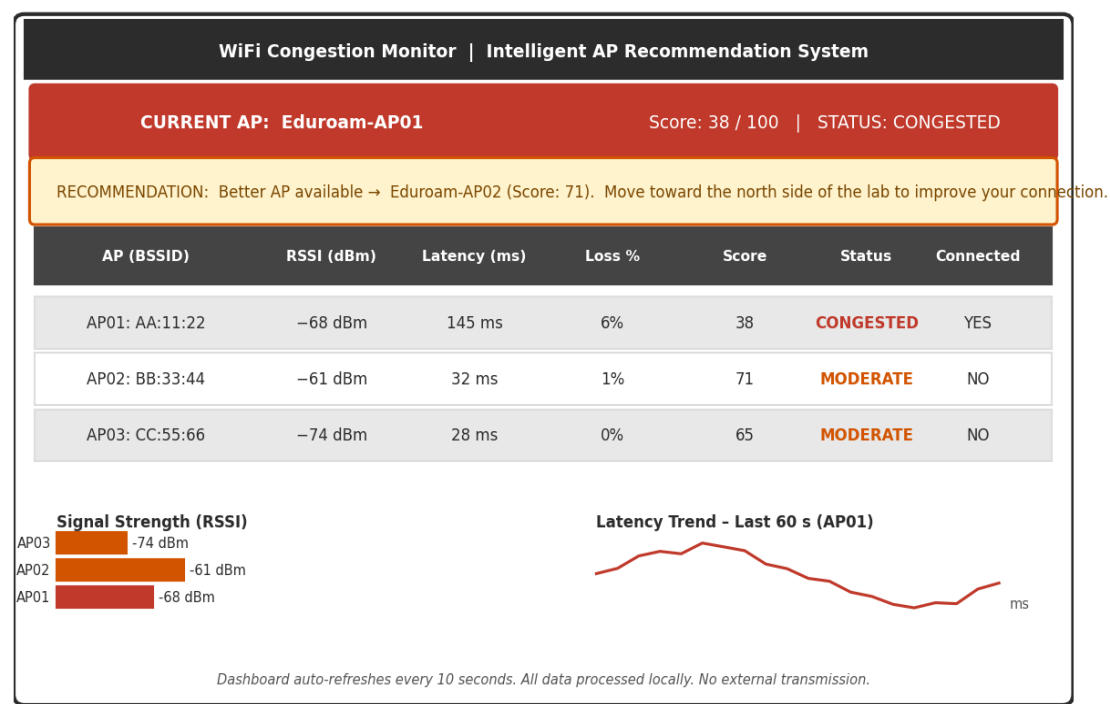


Figure 10 – Dashboard UI Mockup: AP table, status indicators, recommendation box, and latency trend graph

Component	Content	Purpose
Status bar	Current AP name, score, and status badge	Immediate overview of current connection quality
AP comparison table	All detected APs with RSSI, latency, packet loss, score, and status	Comparison of all available connection options
Colour indicators	Green / Orange / Red per AP row	Instant visual communication without numerical understanding required
Recommendation box	Best available AP name and directional guidance message	Actionable advice for the user to improve their connection
Signal bars	Visual bar chart of RSSI levels for all detected APs	Familiar WiFi bar representation for non-technical users
Latency trend graph	Time-series graph of latency over last 60 seconds	Shows whether connection is improving or degrading over time
Timestamp footer	Time of most recent scan	Confirms data is current and monitoring loop is running

Table 12 – Dashboard Components and Their Purpose

11. Multi-Device Support Architecture

The system is designed to support three device types. The same core architecture, scoring algorithm, and dashboard design apply across all devices. The only component that differs between platforms is the WiFi sensing method, as different operating systems provide different levels of access to WiFi scanning data.

Device	Sensing Method	Implementation Stack	Status
Laptop (Windows 10/11)	netsh wlan show networks mode=bssid	Python 3.11, Streamlit, Pandas, CSV	PRIMARY PROTOTYPE
Smartphone (Android)	Android WifiManager API	Android app (Java/Kotlin or Flutter)	Future Implementation
Tablet	Same as smartphone platform API	Same codebase, optimised for larger screen	Future Implementation

Table 13 – Device Support Matrix

12. System Requirements

System requirements define the specific technical behaviours and capabilities the software must deliver to satisfy the user requirements presented in Section 3. These are derived directly from the user requirements.

ID	System Requirement	Linked UR
SR1	The system shall scan all visible WiFi APs and return a list within 5 seconds	UR1, UR7
SR2	The system shall uniquely identify each AP using its BSSID	UR3
SR3	The system shall measure network latency by performing a 10-packet ICMP ping test to the default gateway	UR2, UR7
SR4	The system shall calculate packet loss percentage from the ping test results	UR2
SR5	The system shall compute a quality score (0–100) for each AP using the defined weighted formula	UR2, UR3
SR6	The system shall classify each AP as GOOD, MODERATE, or CONGESTED based on defined thresholds	UR2, UR7
SR7	The system shall display a real-time colour-coded dashboard refreshed every 10 seconds	UR7
SR8	The system shall display a recommendation message when a better AP ($\Delta\text{Score} \geq 20$) is available	UR2, UR3
SR9	The system shall store all measurements and scores locally in a CSV file with ISO-8601 timestamps	UR8
SR10	The system shall not transmit any data to external servers – all processing is local	UR5, UR8

Table 14 – System Requirements Traced to User Requirements

13. Testing Plan and Validation Strategy

The system will be tested in a real-world environment that simulates the high-density university laboratory scenario. Testing is conducted in four phases corresponding to different density and mobility conditions.

Phase	Scenario	Measurement	Expected Outcome
-------	----------	-------------	------------------

Phase 1	Low user density (1–5 devices on one AP)	Score, latency < 30 ms, packet loss < 1%	All APs show GOOD status; no recommendation issued
Phase 2	High user density (15+ devices on one AP)	Score drop, latency > 100 ms	Dashboard shows CONGESTED; recommendation message fires
Phase 3	User moves across room to a better AP	Score change per cycle	Score improves progressively; dashboard updates in real-time
Phase 4	Multiple APs simultaneously visible	AP ranking accuracy	Best AP correctly highlighted; recommendation accurate

Table 15 – D2 Testing Plan

14. Known Limitations

The following limitations are recognised as inherent constraints of the current design and are documented for transparency and academic integrity.

- The system cannot determine the exact physical location of each access point within the building.
- The system cannot force a device to disconnect from one AP and reconnect to another – this decision remains with the operating system.
- Hidden network load information (the number of users connected to each AP) is not accessible without infrastructure-level access.
- RSSI values are affected by physical obstacles, interference from other devices, and temporary RF reflections, which may introduce measurement variability of $\pm 3\text{--}5$ dBm [5, 6].
- Mobile operating systems (particularly iOS) impose restrictions on third-party WiFi scanning APIs, which limits sensing capability on those platforms.
- The weighted scoring formula uses fixed weights selected based on engineering judgement; these weights may be further optimised through real-world testing and machine learning in future work.

PART D3

Complex Engineering Problem

Detailed Design

15. Introduction and Scope of D3

Deliverable D2 established the conceptual architecture of the system and selected the on-device monitoring approach as the most feasible engineering solution. Deliverable D3 now converts that conceptual design into a complete, technically implementable specification. This section answers the question: how exactly does each subsystem work?

D3 provides component-level software design, algorithm pseudocode, data structure definitions, interface specifications, engineering calculations, and a full system integration model. The document is structured such that any qualified engineer could implement the system from this specification without further design decisions.

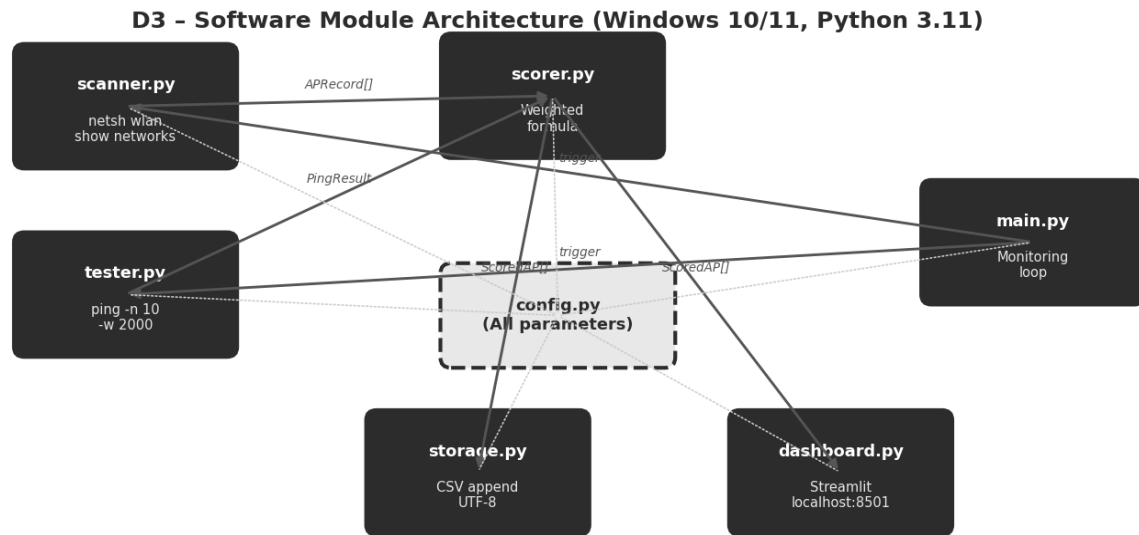
D2 (Conceptual)	D3 (Technical)
High-level block diagram	Module-level software architecture with file names
Subsystem descriptions	Data structures, function signatures, pseudocode
Formula stated	Formula derived, bounds justified, normalisation calculated
CSV storage mentioned	18-column schema defined, write operation specified, capacity calculated
Dashboard described	Component-to-Streamlit-element mapping, refresh mechanism, message templates
Testing phases listed	Unit tests with quantitative pass criteria

Table 16 – D2 vs D3 Design Depth Comparison

16. Refined System Architecture

16.1 Module File Structure

The system is implemented as a Python 3.11 application running on Windows 10/11. It is structured as six independent modules that communicate through well-defined in-memory data structures. All processing occurs locally on the user device with no external network communication between modules [10].



All modules import config.py. No inter-module network calls. All data in-process.

Figure 11 – D3 Software Module Architecture: six Python modules with defined interfaces

Module File	Subsystem	Responsibility
scanner.py	Sensing	Executes netsh wlan show networks mode=bssid and parses raw output into APRecord dataclass instances
tester.py	Network Testing	Executes ping -n 10 -w 2000 <gateway> and computes latency, packet loss, and RTT variance statistics
scorer.py	Scoring	Normalises raw metrics and applies weighted scoring formula; implements recommendation decision logic
storage.py	Data Storage	Appends ScoredAP records to a timestamped CSV file; validates schema on first write
dashboard.py	User Interface	Renders real-time Streamlit dashboard at localhost:8501; reads from Streamlit session state
main.py	Orchestrator	Controls the main monitoring loop; coordinates data flow between all modules every 10 seconds
config.py	Configuration	Centralises all tunable parameters: weights, thresholds, scan interval, normalisation bounds, file paths

Table 17 – Module File Structure and Responsibilities

16.2 Interface Definitions

Interface	From	To	Data Type	Format
I1 – AP List	scanner.py	scorer.py + storage.py	List[APRecord]	Python dataclass list
I2 – Ping Result	tester.py	scorer.py + storage.py	PingResult	Python dataclass

I3 – Scored APs	scorer.py	dashboard.py storage.py	+	List[ScoredAP]	Python dataclass list
I4 – Config Params	config.py	All modules		Constants	Python module import
I5 – CSV Row	storage.py	Filesystem (./logs/)		String	Comma-separated UTF-8

Table 18 – System Interface Definitions

17. Sensing Subsystem – Detailed Design

17.1 OS Command and Parsed Fields

The sensing subsystem invokes the following Windows OS command via Python's subprocess module to retrieve WiFi scan data [7]:

```
import subprocess

result = subprocess.run(
    ['netsh', 'wlan', 'show', 'networks', 'mode=bssid'],
    capture_output=True, text=True, encoding='utf-8'
)
raw_output = result.stdout
```

The raw multi-line text output is parsed using regular expressions to extract the following fields:

Field	netsh Output Line	Data Type	Example
SSID	SSID : Eduroam	String	Eduroam
BSSID	BSSID 1 : AA:11:22:33:44:55	String	AA:11:22:33:44:55
Signal (%)	Signal : 72%	Integer	72
Channel	Channel : 6	Integer	6
Radio type	Radio type : 802.11n	String	802.11n (2.4 GHz)
RSSI (dBm)	Derived from Signal%	Float	-64.0

Table 19 – Sensing Subsystem Parsed Fields

17.2 RSSI Derivation

The Windows netsh utility reports signal quality as a percentage (0–100) rather than directly in dBm units. The RSSI in dBm is derived using the standard Windows NDIS WLAN API conversion formula [7]:

$$\text{RSSI (dBm)} = (\text{Signal\%} / 2) - 100$$

Validation examples:

- Signal% = 100 → RSSI = (100/2) – 100 = –50 dBm (excellent signal, very close to AP)
- Signal% = 72 → RSSI = (72/2) – 100 = –64 dBm (good signal, typical indoor range)
- Signal% = 0 → RSSI = (0/2) – 100 = –100 dBm (no signal, hardware noise floor)

17.3 APRecord Data Structure

```
from dataclasses import dataclass
```

```
@dataclass
class APRecord:
    ssid:      str      # Network name (e.g. 'Eduroam')
    bssid:     str      # Unique AP MAC address (e.g. 'AA:11:22:33:44:55')
    signal_pct: int      # Windows signal quality 0-100
    rssi_dbm:  float    # Derived: (signal_pct / 2) - 100
    channel:   int      # WiFi channel number (1-14 for 2.4 GHz; 36-165 for 5
GHz)
    band:      str      # '2.4GHz' or '5GHz'
    timestamp: str      # ISO-8601 scan timestamp
```

17.4 Hardware Parameters

Parameter	Typical Value	Impact on Design
RSSI measurement range	-30 to -90 dBm	Defines normalisation bounds for signal sub-score
Supported bands	2.4 GHz and 5 GHz	System must handle both frequency bands in parsed output
Scan completion time	2–4 s (OS-controlled)	Sets minimum practical scan interval to 5 seconds
Maximum visible APs	Up to 50 in dense areas	Scoring and display must handle variable-length AP lists
RSSI measurement error	±3–5 dBm (multipath fading) [5, 6]	Score uncertainty ≈ ±5–8 points; mitigated by 20-point switching threshold

Table 20 – WiFi Adapter Hardware Parameters

18. Network Testing Subsystem – Detailed Design

18.1 Ping Command

The network testing subsystem measures the quality of the current active connection by sending ICMP Echo Request packets to the default network gateway. The gateway is identified programmatically using `ipconfig` and used as the ping target [7]. The exact command executed is:

```
import subprocess

result = subprocess.run(
    ['ping', '-n', '10', '-w', '2000', gateway_ip],
    capture_output=True, text=True
)
```

Parameter	Value	Justification
-n 10	10 packets sent	Sufficient sample for statistically reliable packet loss (10% resolution); matches RFC 2544 [9]
-w 2000	2000 ms per-packet timeout	Allows for high-latency congested links; 2 s upper bound is practical for user experience assessment

Packet size	32 bytes (default)	Small probe packets do not themselves cause additional congestion on the tested link
Target	Default gateway (ipconfig)	Closest network hop; measures only local AP-level congestion, not internet backbone latency

Table 21 – Ping Parameters and Engineering Justification

18.2 Derived Performance Metrics

Metric	Calculation	Unit
Mean latency	Arithmetic mean of all received RTT values from 10-packet sample	ms
Minimum latency	Minimum RTT in the sample	ms
Maximum latency	Maximum RTT in the sample	ms
RTT variance	$\text{Var}(\text{RTT}) = (1/n) \times \sum (\text{RTT}_i - \text{RTT})^2$ where $n = 10$	ms ²
Jitter	Standard deviation of RTT sample; approximates RFC 3393 jitter	ms
Packet loss	$(\text{Packets sent} - \text{Packets received}) / \text{Packets sent} \times 100$	%

Table 22 – Derived Network Performance Metrics

18.3 Stability Score Derivation

The Stability metric is derived from the RTT variance. High variance indicates an unstable connection where individual packets experience very different delays, characteristic of a congested AP where the queue length fluctuates [3]:

$$\text{Var}(\text{RTT}) = (1/n) \times \sum (\text{RTT}_i - \text{RTT})^2 \quad [n = 10 \text{ samples}]$$

$$\text{S_stability} = \text{clamp}(100 - (\text{Var} / 25), 0, 100)$$

18.4 PingResult Data Structure

```
@dataclass
class PingResult:
    target_ip: str # IP address of gateway or fallback DNS
    mean_latency_ms: float # Mean RTT across 10 packets (ms)
    min_latency_ms: float # Minimum RTT (ms)
    max_latency_ms: float # Maximum RTT (ms)
    packet_loss_pct: float # Percentage packets lost (0.0 - 100.0)
    variance_ms2: float # RTT variance (ms2) - stability input
    jitter_ms: float # Standard deviation of RTT (ms)
    timestamp: str # ISO-8601 test timestamp
```

19. Intelligent Scoring Subsystem – Detailed Design

19.1 Normalisation Specifications

Each raw metric is linearly normalised to [0, 100] before applying the weights. The normalisation bounds are derived from IEEE 802.11 performance standards and established network quality benchmarks [4, 5].

Signal Strength Normalisation

$$S_signal = clamp((RSSI_dBm + 90) / 60 \times 100, 0, 100)$$

RSSI (dBm)	S_signal	Classification
-30 dBm	100	Excellent – very close to AP
-50 dBm	66.7	Good – typical classroom proximity
-65 dBm	41.7	Fair – usable but moderate attenuation
-75 dBm	25.0	Weak – near edge of reliable coverage [5]
-90 dBm	0	No signal – minimum threshold

Table 23 – Signal Score Normalisation Examples

Latency Normalisation

$$S_latency = clamp((200 - mean_latency_ms) / 190 \times 100, 0, 100)$$

Packet Loss Normalisation

$$S_loss = clamp((20 - packet_loss_pct) / 20 \times 100, 0, 100)$$

Stability Normalisation

$$S_stability = clamp(100 - (variance_ms2 / 25), 0, 100)$$

19.2 Recommendation Decision Logic

The system issues a recommendation only when ALL three conditions are simultaneously satisfied:

- The current AP score is below 75 (not GOOD)
- At least one other visible AP has a score greater than the current AP by at least 20 points
- The recommended AP is on a compatible band (5 GHz preferred over 2.4 GHz for equivalent scores)

The 20-point threshold is an engineering decision: it exceeds the combined measurement uncertainty of ± 5 –12 points identified in Section 21, preventing recommendations based on measurement noise rather than genuine quality differences.

19.3 ScoredAP Data Structure

```
@dataclass
class ScoredAP:
    ap_record:      APRecord      # Full AP scan record from scanner.py
    ping_result:    PingResult     # None if AP not currently connected
    s_signal:       float          # Normalised signal sub-score [0-100]
    s_latency:      float          # Normalised latency sub-score [0-100]
    s_loss:         float          # Normalised loss sub-score [0-100]
    s_stability:    float          # Normalised stability score [0-100]
    total_score:    float          # Weighted composite score [0-100]
    status:         str            # 'GOOD' | 'MODERATE' | 'CONGESTED'
    is_connected:   bool          # True if this is the currently active AP
    is_recommended: bool          # True if recommended by decision logic
```

20. Data Storage Subsystem – Detailed Design

20.1 CSV Schema

Each row in the CSV file represents one AP measurement at one point in time. The schema is defined as follows:

Column Name	Data Type	Source	Example
timestamp	ISO-8601 datetime	System clock	2026-06-10T14:23:05
ssid	String	APRecord	Eduroam
bssid	String (MAC)	APRecord	AA:11:22:33:44:55
channel	Integer	APRecord	6
band	String	APRecord	2.4GHz
rss_i_dbm	Float	APRecord	-62.0
signal_pct	Integer	APRecord	76
mean_latency_ms	Float	PingResult	87.3
packet_loss_pct	Float	PingResult	3.0
variance_ms2	Float	PingResult	412.5
s_signal	Float	ScoredAP	46.7
s_latency	Float	ScoredAP	59.3
s_loss	Float	ScoredAP	85.0
s_stability	Float	ScoredAP	83.5
total_score	Float	ScoredAP	62.4
status	String	ScoredAP	MODERATE
is_connected	Boolean (0/1)	ScoredAP	1
is_recommended	Boolean (0/1)	ScoredAP	0

Table 24 – CSV File Schema (18 Columns)

20.2 File Naming and Write Operation

```
import csv, os
from datetime import datetime

LOG_DIR = './logs/'
os.makedirs(LOG_DIR, exist_ok=True)

# New file per session
filename = f'wifi_log_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv'
filepath = os.path.join(LOG_DIR, filename)

def write_records(scored_aps: list[ScoredAP], filepath: str):
    file_exists = os.path.isfile(filepath)
    with open(filepath, 'a', newline='', encoding='utf-8') as f:
        writer = csv.DictWriter(f, fieldnames=SCHEMA_COLUMNS)
        if not file_exists:
            writer.writeheader() # Write header only on first write
        for ap in scored_aps:
```

```
writer.writerow(ap.to_dict())
```

20.3 Storage Capacity Calculation

- Monitoring cycle interval: 10 seconds
- Maximum APs per scan: 20 (typical dense indoor environment)
- Bytes per CSV row: approximately 200 bytes (18 fields × ~11 characters average)
- Rows per hour: $(3600 / 10) \times 20 = 7,200$ rows/hour
- Storage per hour: $7,200 \times 200$ bytes ≈ 1.4 MB/hour
- Full 8-hour session: $1.4 \times 8 = 11.2$ MB
- Full semester (120 active hours): $1.4 \times 120 \approx 168$ MB → negligible on any modern laptop

21. Engineering Calculations

21.1 Shannon-Hartley Per-User Throughput

The Shannon-Hartley Theorem defines the maximum channel capacity for a 2.4 GHz 802.11n channel of bandwidth $B = 20$ MHz at $\text{SNR} = 20$ dB (linear $\text{SNR} = 100$) [2]:

$$C_{\text{total}} = 20 \times 10^6 \times \log_2(1 + 100) = 20 \times 10^6 \times 6.658 \approx 133 \text{ Mbps}$$

Accounting for 802.11n protocol overhead, CSMA/CA collisions, and guard intervals (efficiency $\approx 45\%$):

$$C_{\text{practical}} \approx 133 \times 0.45 \approx 60 \text{ Mbps} \quad (\text{practical AP channel capacity})$$

Users on AP	Throughput / User	User Experience
5	$60 / 5 = 12.0$ Mbps	Excellent – software downloads complete in seconds
15	$60 / 15 = 4.0$ Mbps	Acceptable – noticeable slowdown during peak
30	$60 / 30 = 2.0$ Mbps	Poor – large downloads take minutes
60	$60 / 60 = 1.0$ Mbps	Severe congestion – system timeout risk
80	$60 / 80 = 0.75$ Mbps	Critical – consistent failures expected

Table 25 – Per-User Throughput vs AP Load (Shannon-Hartley Model) [2]

21.2 M/M/1 Queueing Latency Prediction

Using the M/M/1 model with AP service rate $\mu = 1000$ packets/second and average user rate $\lambda_{\text{user}} = 50$ packets/second [3]:

Users	λ_{total} (pkt/s)	$\mu - \lambda$	W (ms)	Status
5	250	750	1.3 ms	GOOD
10	500	500	2.0 ms	GOOD
15	750	250	4.0 ms	MODERATE

18	900	100	10.0 ms	MODERATE
19	950	50	20.0 ms	CONGESTED
20	1000	$\rightarrow 0$	$\rightarrow \infty$	Saturation – queue collapse

Table 26 – M/M/1 Model: Latency vs User Load [3]

This non-linear relationship justifies weighting latency at 30% – the second highest weight – in the scoring formula. Small increases in user load near saturation cause disproportionately large latency increases [3].

21.3 Measurement Accuracy Analysis

Metric	Error Source	Magnitude	Score Impact
RSSI (dBm)	RF multipath fading; adapter calibration [5, 6]	$\pm 3\text{--}5$ dBm	$\pm 5\text{--}8$ score points
Latency (ms)	OS scheduling jitter; background processes	$\pm 2\text{--}5$ ms	$\pm 1\text{--}3$ score points
Packet loss	Statistical sampling error (n=10)	$\pm 10\%$	± 5 score points
Variance	Small sample noise	$\pm 100\text{--}200$ ms ²	$\pm 4\text{--}8$ score points
Total score	Combined weighted error propagation	$\pm 5\text{--}12$ points	Boundary misclassification risk

Table 27 – Measurement Accuracy Analysis and Score Impact

The combined score uncertainty of $\pm 5\text{--}12$ points is the engineering justification for the 20-point switching threshold. A threshold below the measurement uncertainty would trigger recommendations based on noise rather than genuine quality differences.

21.4 CPU and Memory Footprint

Operation	CPU Time	Memory
netsh WiFi scan	2–4 s (OS I/O wait)	< 1 MB
Text parsing (regex)	< 5 ms	< 1 MB
Ping test (10 × 2 s max)	2–5 s (I/O wait)	< 1 MB
Scoring (20 APs)	< 1 ms	< 1 MB
CSV write (20 rows)	< 5 ms	< 1 MB
Streamlit dashboard	< 100 ms per render	~30 MB (browser tab)
Total per cycle	~4–10 s active	~35 MB resident

Table 28 – CPU and Memory Footprint Estimate

The 35 MB memory footprint is negligible on any laptop with 4+ GB RAM. The application will not perceptibly impact normal student laptop usage.

22. System Integration Design

22.1 Configuration Parameters

Parameter	Default	Description
SCAN_INTERVAL	10 s	Time between complete monitoring cycles
PING_COUNT	10	ICMP packets per ping test
PING_TIMEOUT_MS	2000	Per-packet timeout (ms)
RSSI_MIN / MAX	-90 / -30	Normalisation bounds for signal sub-score
LATENCY_MIN / MAX	10 / 200 ms	Normalisation bounds for latency sub-score
LOSS_MAX	20%	Upper bound for packet loss normalisation
VARIANCE_MAX	2500 ms ²	Upper bound for stability normalisation
W_SIGNAL	0.35	Scoring weight – signal strength
W_LATENCY	0.30	Scoring weight – latency
W_LOSS	0.20	Scoring weight – packet loss
W_STABILITY	0.15	Scoring weight – stability
THRESHOLD_GOOD	75	Score above which AP classified GOOD
THRESHOLD_MOD	50	Score above which AP classified MODERATE
SWITCH_THRESHOLD	20	Min score improvement to trigger recommendation
LOG_DIR	./logs/	Directory for CSV log files
PING_FALLBACK	8.8.8.8	DNS fallback if gateway discovery fails

Table 29 – config.py Parameters with Defaults

22.2 End-to-End Worked Example

The following worked example traces one complete monitoring cycle using realistic values from a congested university laboratory environment.

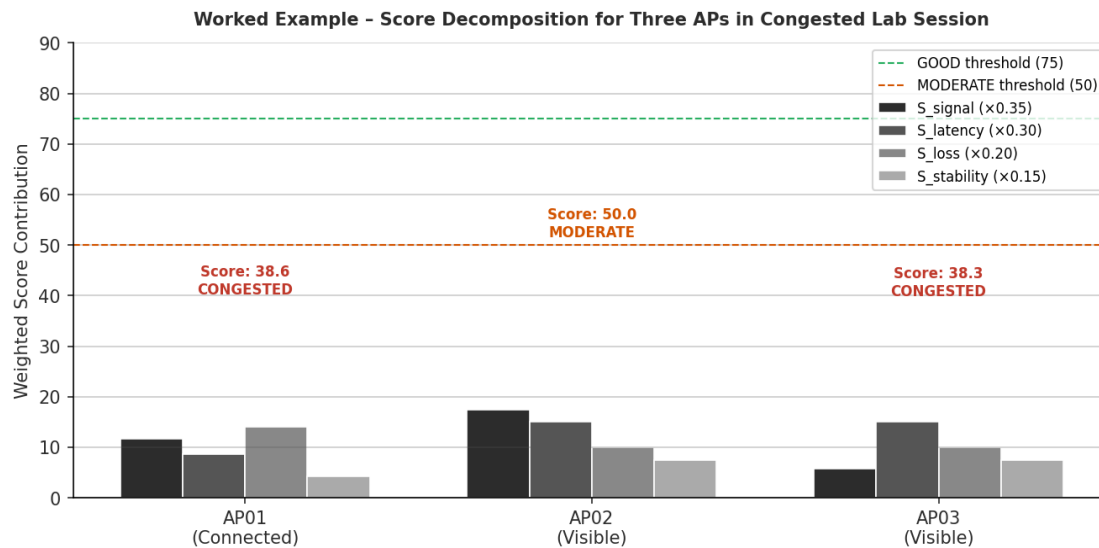


Figure 12 – Worked Example: Score decomposition for three APs showing CONGESTED classification with no recommendation issued

Input Measurements

AP	BSSID (short)	Signal%	RSSI (dBm)	Connected?
AP-01	AA:11:22	60%	-70.0 dBm	YES
AP-02	BB:33:44	80%	-60.0 dBm	NO
AP-03	CC:55:66	40%	-80.0 dBm	NO

Ping result for connected AP-01: mean latency = 145 ms, packet loss = 6%, variance = 1800 ms²

Step 1: Normalisation for AP-01 (connected)

- $S_{\text{signal}} = (-70 + 90) / 60 \times 100 = 20/60 \times 100 = 33.3$
- $S_{\text{latency}} = (200 - 145) / 190 \times 100 = 55/190 \times 100 = 28.9$
- $S_{\text{loss}} = (20 - 6) / 20 \times 100 = 14/20 \times 100 = 70.0$
- $S_{\text{stability}} = 100 - (1800 / 25) = 100 - 72 = 28.0$

Step 2: Score Calculation for AP-01

$$\text{Score} = 0.35 \times 33.3 + 0.30 \times 28.9 + 0.20 \times 70.0 + 0.15 \times 28.0 = 11.7 + 8.7 + 14.0 + 4.2 = 38.6 \rightarrow \text{CONGESTED}$$

Step 3: AP-02 and AP-03 (not connected – signal score only, other sub-scores default to 50)

$$\text{Score}_{\text{AP02}} = 0.35 \times 50.0 + 0.30 \times 50.0 + 0.20 \times 50.0 + 0.15 \times 50.0 = 50.0 \rightarrow \text{MODERATE}$$

$$\text{Score}_{\text{AP03}} = 0.35 \times 16.7 + 0.30 \times 50 + 0.20 \times 50 + 0.15 \times 50 = 38.3 \rightarrow \text{CONGESTED}$$

Step 4: Recommendation Decision

- Current AP (AP-01) score: 38.6 – below GOOD threshold (75) ✓
- Best alternative: AP-02, score 50.0 | Delta = 50.0 – 38.6 = 11.4
- Delta (11.4) is BELOW switching threshold (20) ✗

- Result: No recommendation issued. Dashboard message: 'Current AP congested. No significantly better AP detected. Consider moving to a different area.'

23. User Interface Subsystem – Detailed Design

23.1 Implementation: Streamlit

The dashboard is implemented using Streamlit, a Python framework that renders interactive web dashboards directly from Python code accessible via the browser at <http://localhost:8501>. No HTML, CSS, or JavaScript is required. Streamlit is installed via pip without administrator rights and is the chosen framework because it provides native support for real-time refresh (`st.rerun()`), colour-coded metric boxes (`st.metric`), and inline chart rendering (`st.line_chart`, `st.bar_chart`) [10].

23.2 Streamlit Component Mapping

Dashboard Component	Streamlit Element	Content
Page header	<code>st.title + st.caption</code>	System name, scan count, last updated timestamp
Current AP status	<code>st.metric</code> (3 columns)	AP name, score, status badge
Status alert	<code>st.success</code> / <code>st.warning</code> / <code>st.error</code>	Colour-coded GOOD / MODERATE / CONGESTED alert
AP table	<code>st.dataframe</code> with Styler	All APs: BSSID, RSSI, latency, loss, score, status
Recommendation	<code>st.info</code> or <code>st.warning</code> block	Recommended AP name, score delta, directional advice
Signal bar chart	<code>st.bar_chart</code>	RSSI values for all detected APs
Latency trend	<code>st.line_chart</code>	30-second rolling latency window for connected AP
Raw data expander	<code>st.expander + st.dataframe</code>	Full PingResult and APRecord on demand

Table 30 – Streamlit Component Mapping

23.3 Recommendation Message Templates

Condition	Message Displayed
Current AP is GOOD	Connected to best available AP. Score: {score}/100. No action needed.
Better AP available ($\Delta\text{Score} \geq 20$)	Better connection available: {ssid} ({bssid}) – Score: {alt_score}/100. Move toward that area to improve your speed. Improvement: +{delta} points.
Congested, no better alternative	Current AP is congested (Score: {score}/100). No significantly better AP detected. Consider moving to a different area of the building.
Scan returned no APs	No WiFi networks detected. Please verify that WiFi is enabled on this device.

Table 31 – Recommendation Message Templates

24. Unit and Integration Testing Plan

24.1 Unit Tests

Test ID	Module	Description	Pass Criterion
UT-01	scanner.py	Execute netsh scan; verify ≥ 1 APRecord with valid BSSID returned	List length ≥ 1 ; BSSID matches MAC regex <code>^([0-9A-Fa-f]{2}):([0-9A-Fa-f]{2}):([0-9A-Fa-f]{2}):([0-9A-Fa-f]{2}):([0-9A-Fa-f]{2}):([0-9A-Fa-f]{2})\$</code>
UT-02	scanner.py	Inject Signal% = 72; verify derived RSSI = -64.0 dBm	RSSI within ± 0.1 dBm of -64.0
UT-03	tester.py	Execute ping to 8.8.8.8 n=10; verify PingResult populated	mean_latency_ms > 0; packet_loss_pct in [0, 100]
UT-04	scorer.py	Input RSSI=-45, latency=30 ms, loss=0%, variance=0; verify score ≥ 75	total_score ≥ 75 ; status = 'GOOD'
UT-05	scorer.py	Input RSSI=-88, latency=195 ms, loss=18%, variance=2400; verify score < 50	total_score < 50; status = 'CONGESTED'
UT-06	scorer.py	Two APs: current score=40, alternative score=65 (delta=25); verify recommendation fires	is_recommended = True on alt AP
UT-07	scorer.py	Two APs: current score=40, alternative score=59 (delta=19); verify no recommendation	is_recommended = False on all APs
UT-08	storage.py	One monitoring cycle; verify CSV created with correct 18-column header and rows	Header matches Table 24 schema; row count = AP count
UT-09	storage.py	Verify timestamp field parses as valid ISO-8601	<code>datetime.fromisoformat(timestamp)</code> succeeds without exception

Table 32 – Unit Test Specifications with Pass Criteria

24.2 Integration Tests

Test ID	Scenario	Method	Pass Criterion
IT-01	Low density: 1–3 devices	Monitor during off-peak hours; 10 cycles	Score ≥ 75 in ≥ 8 of 10 cycles; no recommendation
IT-02	High density: 15+ devices	Monitor during lab session; 10 cycles	Score < 50 detected; CONGESTED shown; recommendation fires
IT-03	Physical movement test	Move laptop across room; record score per cycle	Score changes ≥ 10 points over 3+ cycles; trend visible in CSV
IT-04	Multi-AP comparison	Scan in area with ≥ 3 visible APs	All APs shown in table with distinct scores; best AP highlighted

IT-05	30-minute continuous operation	Leave system running; verify no crashes	No unhandled exceptions; CSV grows at expected rate; refresh every 10 s
-------	--------------------------------	---	---

Table 33 – Integration Test Specifications

25. Known Limitations and Future Work

25.1 Known Limitations

Limitation	Engineering Root Cause	System Impact
Ping data available for connected AP only	Windows ICMP tests not possible through non-connected interfaces	Unconnected APs scored on signal alone; ranking uncertainty introduced
RSSI noise of ± 3 –5 dBm [5, 6]	RF multipath fading and adapter calibration variation	Score uncertainty ± 8 points; mitigated by 20-point threshold
netsh scan takes 2–4 s (OS-controlled)	Windows WLAN service controls scan timing internally	Minimum cycle is 5–10 s; real-time reaction to sudden congestion limited
Hidden AP load data unavailable	Client count not broadcast in 802.11 beacon frames [4]	Congestion inferred indirectly from latency/loss; no direct user count
Windows-only prototype	netsh is Windows-specific [7]	Cross-platform port requires iwlist (Linux) or airport (macOS)

Table 34 – Known Limitations and Root Causes

25.2 Future Work

- Machine learning scoring model: replace fixed weights with a Random Forest model trained on labelled AP quality data collected during testing.
- Android mobile prototype: implement sensing using Android WifiManager API to extend to smartphones and tablets.
- Multi-node architecture: deploy Raspberry Pi sensing nodes in multiple rooms, aggregating data to a central dashboard via MQTT protocol.
- Automatic AP switching: investigate 802.11r BSS Fast Transition support on campus APs to enable seamless roaming triggered by the recommendation engine [4].
- Live user count estimation: correlate latency patterns with user density using M/M/1 model inversion to estimate connected users per AP [3].

References

- [1] Engineering Council of South Africa (ECSA), Accreditation Standard for Engineering Programmes, Pretoria: ECSA, 2019.
- [2] C. E. Shannon, "A Mathematical Theory of Communication," Bell System Technical Journal, vol. 27, no. 3, pp. 379–423, July 1948.
- [3] L. Kleinrock, Queueing Systems Volume 1: Theory. New York: John Wiley & Sons, 1975.
- [4] IEEE 802.11-2020, IEEE Standard for Information Technology – Telecommunications and Information Exchange Between Systems – Local and Metropolitan Area Networks – Specific Requirements – Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications. New York: IEEE, 2020.
- [5] A. Goldsmith, Wireless Communications. Cambridge: Cambridge University Press, 2005.
- [6] D. Tse and P. Viswanath, Fundamentals of Wireless Communication. Cambridge: Cambridge University Press, 2005.
- [7] Microsoft Corporation, "NDIS_WLAN_ASSOCIATION_ATTRIBUTES structure (windot11.h)," Windows Hardware Developer Documentation, 2023. [Online]. Available: <https://docs.microsoft.com/en-us/windows-hardware/drivers/ddi/windot11/>
- [8] V. Jacobson, "Congestion avoidance and control," in Proc. ACM SIGCOMM, Stanford, CA, 1988, pp. 314–329.
- [9] RFC 2544, "Benchmarking Methodology for Network Interconnect Devices," Internet Engineering Task Force (IETF), Feb. 1999.
- [10] Streamlit Inc., Streamlit Documentation, Version 1.32, 2024. [Online]. Available: <https://docs.streamlit.io>